Codebase Guide: SEAS-MFEM BP5

Auto-generated by chunhui-codebase-guide on 2026-03-30. Source: `/Users/chunhuizhao/projects/seas-mfem/miniapps/seas/`
Problem context: SCEC SEAS BP5-QD (3D strike-slip earthquake cycles)

This guide traces the full derivation from the continuous PDE through the DG weak form to the discrete matrix equations and their implementation. Every code reference points to a specific file and line range.

Table of Contents

---

1. Repository Layout and Call Graph

1.1 File Organization

Path	Purpose
`pseas.cpp`	Main driver (currently BP2; BP5 driver is in tests/verification/)
`tests/verification/bp5_verification_test.cpp`	**Primary BP5 driver**: mesh loading, component wiring, time loop
`config/bp5_params.hpp`	All BP5 physical parameters, spatial parameter functions a(x2,x3), initial conditions
`domain/elasticity_operator.hpp`	3D DG elasticity: stiffness assembly, Solve(), ComputeTraction()
`domain/domain_operator.hpp`	Abstract base class for domain operators

Path	Purpose
`integrator/dg_elasticity_ip_domain_integrator.hpp`	IP-DG face integrator: K assembly, slip RHS, traction recovery
`integrator/dg_elasticity_br2_integrator.hpp`	BR2-DG face integrator (alternative to IP)
`integrator/dg_elasticity_ip_legacy_integrator.hpp`	Legacy IP integrator (validation only)
`fault/rate_state_fault.hpp`	Fault ODE operator: ComputeRHS() couples traction to slip rate
`fault/fault_geometry.hpp`	Extracts fault DOFs, precomputes spatially varying a, L, tau_pre
`fault/fault_basis.hpp`	Per-face local coordinate frames (normal, dip, strike)
`fault/face_quadrature.hpp`	Face basis functions for multi-DOF fault discretization
`friction/dieterich_ruina.hpp`	Regularized Dieterich-Ruina friction: f(V,psi), SolveSlipRate, SolveSlipRateVectorPsi
`friction/state_evolution.hpp`	Aging law, slip law: dpsi/dt
`friction/friction_law.hpp`	Abstract friction interface
`solver/seas_operator.hpp`	Top-level ODE operator: couples domain + fault, implements Mult()
`solver/time_stepper.hpp`	Dormand-Prince RK45 adaptive integrator
`io/bp5_parallel_output.hpp`	Distributed probe output (SCEC fltst format)
`io/probe_output.hpp`	Single-station file writer
`io/checkpoint.hpp`	Checkpoint/restart
`common/seas_types.hpp`	Serial/parallel type aliases
`common/mpi_context.hpp`	MPI wrapper (GlobalMax, GlobalSumInt, etc.)

1.2 Call Graph: One ODE Evaluation

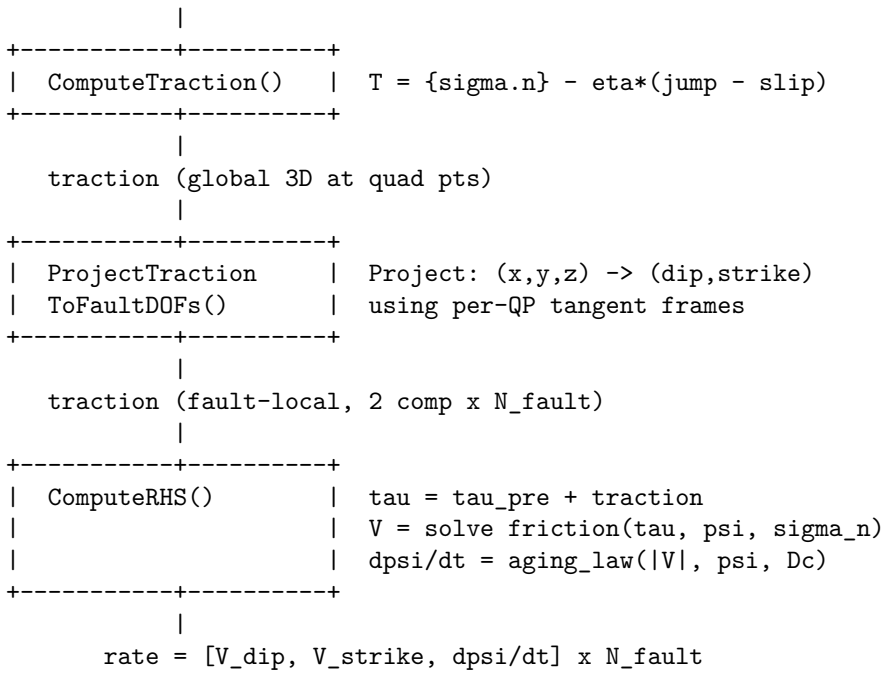
Each call to `SEASQuasiDynamicOperator::Mult(state, rate)` performs one complete physics evaluation. The ODE integrator calls this 7 times per Dormand-Prince step (6 stages + FSAL reuse).

```
DormandPrinceRK45::Step()                          [time_stepper.hpp:227]
  |
  +-- op.Mult(state, rate)                         [seas_operator.hpp:244]
       |
       +-- fault_->GetSlip(state, slip_)           [rate_state_fault.hpp]
       |     Extract slip components from state vector.
       |     State layout: [s_dip_0, s_strike_0, psi_0, s_dip_1, ...]
       |
       +-- domain_->Solve(t, slip_, u_gf_)         [elasticity_operator.hpp:2772]
       |     |
       |     +-- AssembleStiffness()                [elasticity_operator.hpp:962]
       |     |     Build K once: volume + face integrators
```

```
|     |     (cached across all subsequent Solve calls)
|     |
|     +-- AssembleSlipContributionIP()  [elasticity_operator.hpp:1163]
|     |     Build RHS b_slip from fault slip (every call)
|     |
|     +-- AssembleDirichletLoading()    [elasticity_operator.hpp:1865]
|     |     Build RHS b_dir from plate rate BCs (every call)
|     |
|     +-- solver_->Mult(B_, X_)         MUMPS/CG+AMG solve: K u = b
|
+-- domain_->ComputeTraction(u, slip, traction)
|     [elasticity_operator.hpp:3176]
|     Evaluate T = {sigma.n} - penalty*(jump - slip) at fault faces
|     Project to fault-local (dip, strike) components
|
+-- fault_->ComputeRHS(traction, state, rate)
      [rate_state_fault.hpp:363]
      For each fault DOF:
        tau_total = tau_pre + traction
        V = SolveSlipRateVectorPsi(tau_total, psi, sigma_n, eta, a)
        rate = [V_dip, V_strike, dpsi/dt]
```

1.3 Data Flow Diagram

```
            state = [slip_dip, slip_strike, psi] x N_fault
                 |
                 v
       +------------------+
       | GetSlip()        |  Extract slip (2 components x N_fault)
       +------------------+
                 |
          slip (fault-local)
                 |
    +------------+----------+
    |  FaultBasis::         |  Embed: (dip,strike) -> (x,y,z)
    |  EmbedSlipQP()        |  using per-face tangent frames
    +------------+----------+
                 |
           slip (global 3D)
                 |
    +------------+----------+
    |  ElasticityDomain     |
    |  Operator::Solve()    |  K u = b(slip, Dirichlet_BC)
    +------------+----------+
                 |
          u (displacement)
```



---

2. Governing Physics: BP5-QD

2.1 Problem Statement

BP5-QD is a 3D quasi-dynamic earthquake cycle problem. An elastic half-space (x1 in [-infinity, +infinity], x2 in [-infinity, +infinity], x3 in [0, infinity] depth-positive-down) contains a vertical planar fault at x1=0. The fault supports rate-and-state friction. Far-field tectonic loading drives right-lateral strike-slip at plate rate Vp.

Coordinate mapping (SCEC abstract to code/mesh): - SCEC x1 (fault-normal) = code Y (fault at Y=0) - SCEC x2 (along-strike) = code X - SCEC x3 (depth, positive down) = code -Z (Z=0 at surface, negative downward)

2.2 Governing Equations

Momentum balance (quasi-static, no inertia):

```
div(sigma) = 0     in Omega
```

where sigma is the Cauchy stress tensor for a linear isotropic elastic solid:

```
sigma = lambda * tr(epsilon) * I + 2 * mu * epsilon
epsilon = (1/2) * (grad(u) + grad(u)^T)
```

Boundary conditions: - **Dirichlet (far-field):** u_X = sgn(Y) * Vp * t / 2 on far-field faces (attr 5) - **Natural (free surface + bottom):** sigma . n = 0 on top (Z=0) and bottom (attr 1) - **Fault (interior interface at Y=0):** slip = [[u]] is prescribed by the ODE

Fault stress balance (quasi-dynamic with radiation damping):

```
tau_total = tau_pre + tau_elastic
|tau_total| = sigma_n * f(|V|, psi) + eta * |V|
V is anti-parallel to tau_total
```

where eta = mu/(2*cs) is the radiation damping coefficient.

State evolution (aging law in psi-space):

```
psi = f0 + b * ln(V0 * theta / Dc)
dpsi/dt = (b * V0 / Dc) * exp(-psi/b) - (b / Dc) * |V|
```

which is the transformed form of dtheta/dt = 1 - |V|*theta/Dc.

2.3 Key Parameters

Symbol	Code variable	File	Value	Meaning
rho	`BP5Params::rho`	`config/bp52_params.hpp`	2670 kg/mˆ3	Rock density
cs	`BP5Params::cs`	`config/bp53_params.hpp`	3464 m/s	Shear wave speed
mu	`BP5Params::mu()`	`config/bp5rha*cs^2_params.hpp`	= 32.04 GPa	Shear modulus
lambda	`BP5Params::lambda()`	`config/bp52_params.hpp`	rho*(cs^2 - 2*nu) = 32.04 GPa	First Lame parameter
nu	`BP5Params::nu`	`config/bp50_params.hpp`	0.25	Poisson's ratio
eta	`BP5Params::eta()`	`config/bp5mu/(2*cs)_params.hpp`	= *4.627 MPa*s/m	Radiation damping
sigma_n	`BP5Params::sigma_n`	`config/bp52_params.hpp`	25 MPa	Effective normal stress
Vp	`BP5Params::Vp`	`config/bp51_params.hpp`	1.0 m/s	Plate rate
V0	`BP5Params::V0`	`config/bp51e-6_params.hpp`	1e-6 m/s	Reference slip rate
f0	`BP5Params::f0`	`config/bp50_params.hpp`	0.6	Reference friction
a	`BP5Params::a_of_x2_x3()`	`config/bp50.004+0.04_params.hpp`	0.004 + 0.04 * ...	Direct effect (spatially varying)

Symbol	Code variable	File	Value	Meaning
b	`BP5Params::b`	`config/bp5_params.hpp:73`	0.03	State evolution effect
Dc (L)	`BP5Params::L_of_x2_x3()`	`config/bp5_params.hpp:238`	12–14 m	Critical slip distance

2.4 Entry Point

The BP5 simulation is launched from `tests/verification/bp5_verification_full.cpp`.

```
bp5_verification_full.cpp::main()
  |
  +-- Create/load 3D mesh (Gmsh .msh or inline hex)
  +-- ElasticityDomainOperator<ParMesh> domain(mesh, order, lambda, mu, ...)
  +-- FaultGeometry<ParMesh> fault_geom(domain, bp5_params, &mpi)
  +-- DieterichRuinaFriction friction(fc)
  +-- AgingLaw aging   (operates in psi-space via AgingLawPsi)
  +-- RateStateFaultOperator<ParMesh, 2> fault_op(&fault_geom, &friction, &aging, bp5_params
  +-- PBP5SEASOp seas_op(&domain, &fault_op, &mpi)
  +-- seas_op.SetInitialCondition(state)
  +-- DormandPrinceRK45 ode_solver
  +-- Time loop: ode_solver.Step(seas_op, state, t, dt)
```

---

3. Continuous Weak Form

3.1 Deriving the Weak Form

Start with the strong form of momentum balance:

```
div(sigma(u)) = 0     in Omega
```

Multiply by a vector test function w, integrate over the domain Omega (split into elements K), and integrate by parts element-wise:

```
sum_K [ integral_K sigma(u) : epsilon(w) dV
      - integral_{dK} (sigma(u) . n) . w dS ] = 0
```

The boundary integrals split into: 1. **Interior faces** (between elements): DG flux terms 2. **Fault faces** (at Y=0): slip boundary condition 3. **Dirichlet boundary faces**: plate-rate loading 4. **Natural boundary faces**: zero traction (free surface)

3.2 Interior Face Terms (DG Skeleton)

On each interior face F shared by elements K- and K+, the SIPG weak form introduces three terms:

```
a_F(u, w) = - integral_F {sigma(u).n} . [[w]] dS          (consistency)
           + epsilon * integral_F {sigma(w).n} . [[u]] dS  (symmetry, eps=-1 for SIPG)
           + integral_F penalty * [[u]] . [[w]] dS          (penalty/stabilization)
```

where: - `{v} = (v- + v+) / 2` is the average - `[[v]] = v- - v+` is the jump (normal from K- to K+) - `sigma(v) . n` is the traction operator: `[sigma(v).n]_i = lambda * (div v) * n_i + mu * (dv_i/dx_j * n_j + dv_j/dx_i * n_j)` - `penalty` is the IP penalty parameter (see Section 4.2)

3.3 Fault Face Terms

On fault faces, the jump is prescribed: `[[u]] = delta` (the slip vector). This splits the bilinear form: the unknown part of [[u]] goes into K, and the known slip delta goes into the RHS vector b.

```
a_fault(u, w) = [same 3 DG terms as interior faces]

But [[u]] is decomposed:
  [[u]] = (unknown jump enforced by K) + delta (prescribed slip in RHS)
```

In practice, the stiffness matrix K treats fault faces identically to other interior faces. The RHS b encodes the prescribed slip:

```
b_slip[w] = - epsilon * integral_F {sigma(w).n} . delta dS
          + integral_F penalty * delta . w dS
```

3.4 Dirichlet Boundary Terms

On Dirichlet faces, the solution is prescribed: u = u_D. The DG boundary contribution is:

```
b_dir[w] = - epsilon * integral_F sigma(w).n . u_D dS
          + integral_F penalty * u_D . w dS
```

where u_D = (sgn(Y) * Vp * t / 2, 0, 0) for BP5 (right-lateral strike-slip loading in the X direction).

---

4. DG Discretization: Weak Form to Matrix Equations

4.1 From Continuous to Discrete

The DG finite element space uses discontinuous polynomials of order p on each element. For BP5: vector-valued with 3 components (u_x, u_y, u_z), `Ordering::byNODES`.

FE Space setup (`elasticity_operator.hpp:373-379`):

```
fec_ = std::make_unique<DG_FECollection>(order_, 3, BasisType::GaussLobatto);
fes_ = std::make_unique<FESpaceType>(&mesh_, fec_.get(), 3, Ordering::byNODES);
```

The discrete system is: $\mathbf{K}\ \mathbf{u} = \mathbf{b}$, where: - $\mathbf{K}$ = stiffness matrix from volume + face integrals (assembled once, cached) - $\mathbf{b}$ = RHS from slip BCs + Dirichlet loading (reassembled every Solve call) - $\mathbf{u}$ = displacement DOF vector

4.2 Volume Term -> ElasticityIntegrator

The volume integral:

```
integral_K sigma(u) : epsilon(w) dV
```

maps directly to MFEM's built-in `ElasticityIntegrator`:

```
// elasticity_operator.hpp:979-980
cached_a_->AddDomainIntegrator(
   new ElasticityIntegrator(lambda_coeff_, mu_coeff_));
```

This contributes the block-diagonal part of K (no inter-element coupling).

4.3 Interior Face Terms -> IP or BR2 Integrator

The code supports two DG penalty methods.

4.3.1 IP Method (Interior Penalty) The IP combined integrator (`dg_elasticity_ip_combined_integrator.hpp`) assembles all three face terms (consistency + symmetry + penalty) in one pass. The face matrix block between test element a and trial element b is:

```
elmat[(a,k,i), (b,l,u)] += w_q * [
    c0 * traction(phi_k e_i, n_q) * shape_b[l] * delta_{u,*}    (consistency)
  + c1 * traction(phi_l e_u, n_q) * shape_a[k] * delta_{i,*}    (symmetry)
  + c2 * shape_a[k] * shape_b[l] * delta_{iu}                    (penalty)
]
```

where the traction operator for a basis function phi_k in direction e_i is:

```
traction(phi_k e_i, n)_u = lambda * (d phi_k / dx_i) * n_u
                         + mu * (delta_{iu} * (grad phi_k . n)
                               + (d phi_k / dx_u) * n_i)
```

This is implemented in `AssembleFaceBlock()` (line ~170 of the combined integrator).

The coefficients c0, c1, c2 depend on the block position:

Block	c0	c1	c2	Meaning
(0,0) same side	-0.5	eps/2	+penalty	test=elem1, trial=elem1
(0,1) cross	-0.5	-eps/2	-penalty	test=elem1, trial=elem2
(1,0) cross	+0.5	+eps/2	-penalty	test=elem2, trial=elem1
(1,1) same side	+0.5	-eps/2	+penalty	test=elem2, trial=elem2

(Source: `dg_elasticity_ip_combined_integrator.hpp:108-166`)

IP Penalty parameter (`dg_elasticity_ip_combined_integrator.hpp:796-815`):

```
c_N = p * (p + dim - 1) / dim
ratio = (dim * lambda + 2*mu)^2 / (2*mu)
penalty_face = penalty_factor * (dim+1) * c_N * (dim * |n| / detJ) * ratio
```

For interior faces, the penalty is the average of both sides divided by 4. This matches Tandem's `estimateD()` formula.

4.3.2 BR2 Method (Bassi-Rebay 2) The BR2 integrator (`dg_elasticity_br2_integrator.hpp`) replaces the penalty term with a lifting operator. The bilinear form is:

```
a_F(u, w) = - integral_F {sigma(u).n} . [[w]] dS
          + epsilon * integral_F {sigma(w).n} . [[u]] dS
          + sigma_BR2 * integral_F C:R_h(u) : R_h(w) dS
```

where R_h is the BR2 lifting operator defined via: 1. Compute face integral: `IntFace[m,s] = sum_q shape[m,q] * source[l,q] * n[s,q] * w[q]` 2. Apply mass inverse: `Lift[l,s,m] = 0.5 * Minv[m,o] * IntFace[l*dim+s, o]` 3. Contract with elasticity tensor via `test_normal`: `test_normal(i,u,s) = lambda * delta_{us} * n_i + mu * (delta_{iu} * n_s + delta_{is} * n_u)` 4. Evaluate lifted function: `L_q[l,i,u,q] = 0.5 * sum_{elem,s,m} test_normal * shape[m,q] * Lift[m,s]`

The BR2 penalty parameter is simply `sigma = num_faces_per_element`: - 4 for tetrahedra (dim+1) - 6 for hexahedra (2*dim)

The mass matrix inverse is precomputed per element in `PrecomputeMassInverse()` (`elasticity_operator.hpp:861-956`).

Code path (`elasticity_operator.hpp:982-1001`):

```
if (method_ == DGMethod::BR2)
{
   cached_a_->AddInteriorFaceIntegrator(
      new DGElasticityBR2Integrator(
         lambda_coeff_, mu_coeff_, epsilon_, elem_mass_inv_, 3));
   cached_a_->AddBdrFaceIntegrator(
      new DGElasticityBR2BoundaryIntegrator(
         lambda_coeff_, mu_coeff_, epsilon_, elem_mass_inv_, 3),
      dirichlet_bdr_marker_);
}
```

4.4 Quadrature

All face integrals use quadrature order `2*p + 1` (matching Tandem's MinQuadOrder), where p is the polynomial order of the DG space. This is exact for the product of two shape function gradients on the face.

5. Assembly Pipeline: K, b, and the Linear Solve

5.1 Stiffness Matrix K

K is assembled once in `AssembleStiffness()` (`elasticity_operator.hpp:962-1157`) and cached for all subsequent Solve calls. The stiffness matrix is SPD for SIPG (epsilon=-1) with sufficient penalty.

```
K = K_vol + K_interior_faces + K_boundary_faces
```

Assembly steps: 1. `AddDomainIntegrator(ElasticityIntegrator)` – volume terms 2. `AddInteriorFaceIntegrator(DGElasticityIPCombinedIntegrator)` – ALL interior faces (including fault) 3. `AddBdrFaceIntegrator(...)` – Dirichlet boundary faces 4. `cached_a_->Assemble()` and `Finalize()` 5. `ParallelAssemble(cached_Ah_)` – creates HypreParMatrix

Key point: Fault faces are interior faces in MFEM and contribute to K just like any other interior face. The fault slip enters only through the RHS vector b.

5.2 RHS Vector b

The RHS is rebuilt every Solve call (`elasticity_operator.hpp:2780-2827`):

```
b = b_slip + b_dirichlet
```

5.2.1 Slip RHS (b_slip) Assembled in `AssembleSlipContributionIP()` (`elasticity_operator.hpp:1163-1266`).

For each fault face: 1. **Transform slip from fault-local to global:** The 2-component (dip, strike) slip is embedded into 3D using `FaultBasis::EmbedSlipQP()`, producing `delta_u[3]` at each quad point.

2. **Sign correction:** The slip sign depends on which side of the fault the mesh normal points. `sign = basis.sign_flipped ? +1 : -1`.

3. **Assemble RHS via combined integrator:**

```
slip_integrator.AssembleSlipFaceRHS(
   *fe1, *fe2, *FTr, delta_u_quad, elvec1, elvec2);
```

The `AssembleSlipFaceRHS` method (`dg_elasticity_ip_combined_integrator.hpp:210-340`) computes two contributions per element:

```
b_k^i += c1 * sum_u traction(phi_k e_i, n)_u * f_u * w_q / detJ    (symmetry data)
       + c2 * phi_k * f_i * w_q                                      (penalty data)
```

where `c1 = epsilon/2` and `c2 = +penalty` for element 1, `-penalty` for element 2.

Consistency guarantee: The slip RHS uses the SAME integrator class (`DGElasticityIPCombinedIntegrator`) and the SAME penalty formula as the stiffness matrix K. This K-b consistency is critical for correctness.

5.2.2 Dirichlet RHS (b_dirichlet) Assembled in `AssembleDirichletLoading()` (`elasticity_operator.hpp:1865`).

The Dirichlet value is u_D = (sgn(Y) * Vp * t / 2, 0, 0), applied on boundary faces with attribute 5 (far-field walls). The same symmetry + penalty pattern is used, with full (not halved) coefficients since boundary faces have only one adjacent element.

5.3 Linear Solve

System: K u = b, where K is a sparse SPD matrix (for SIPG with DG).

Solver options (selected via `SolverType` enum, `elasticity_operator.hpp:36`):

Solver	Code path	Use case
`MUMPS_BLR` (default)	`elasticity_operator.hpp:1041-1049`	Direct solver with BLR compression. Robust, moderate memory.
`MUMPS`	`elasticity_operator.hpp:1033-1040`	Direct solver, full factorization
`CG_AMG`	`elasticity_operator.hpp:1131-1184`	CG + BoomerAMG. Iterative, low memory.
`GMRES_AMG`	`elasticity_operator.hpp:1102-1128`	For DG at high p where AMG is non-SPD
`GMRES_BlockILU`	`elasticity_operator.hpp:1084-1100`	Block-ILU preconditioner (DG-natural)

The solve happens in `Solve()` at `elasticity_operator.hpp:2772`:

```
void Solve(real_t time, const Vector &slip_bc, GridFuncType &displacement)
{
   if (!stiffness_assembled_) { AssembleStiffness(); }   // K: assembled once
   LinFormType b(fes_.get());
   b.Assemble();
   AssembleSlipContributionIP(rhs, slip_bc);   // b_slip: every call
   AssembleDirichletLoading(rhs, time);         // b_dir: every call
   solver_->Mult(B_, X_);                       // Solve K u = b
}
```

6. Fault Interface: Slip to Traction

6.1 Fault Face Detection

Fault faces are identified by mesh boundary element attributes. Tandem's Gmsh mesh tags faces with Physical Surface attributes: - **Tag 1** = Natural (top + bottom) - **Tag 3** = Fault (Y=0 interior) - **Tag 5** = Dirichlet (far-field)

`BuildFaultTaggedFaces()` (`elasticity_operator.hpp:421-498`) maps each attr-3 boundary element to its face index using `GetBdrElementFaceIndex()`. A coordinate-based fallback exists for meshes without tags.

6.2 Fault Coordinate Frame

Each fault face gets a local coordinate frame: (dip, strike, normal).

Algorithm (matching Tandem `Curvilinear::facetBasis`, in `fault_basis.hpp:44-100`): 1. Get raw face normal from `CalcOrtho(face_jacobian, n_raw)` 2. Orient: if `dot(n_raw, ref_normal) < 0`, flip. `ref_normal = (0, -1, 0)`. 3. Normalize: `n = n_raw / |n_raw|` 4. `strike = normalize(up x n)`, where `up = (0, 0, 1)` 5. `dip = strike x n` 6. `tangent1 = dip`, `tangent2 = strike`

For higher-order elements, per-quadrature-point tangent frames are computed (`ComputeQPBasis`, `fault_basis.hpp:112-138`) since the normal varies across the face for curved meshes.

6.3 Traction Recovery

After solving K u = b, the traction on each fault face is recovered in `ComputeTractionImpl()` (`elasticity_operator.hpp:3176`).

IP method (primary path for BP5):

The traction at each quad point has two parts:

```
T(q) = T_stress(q) + T_correction(q)
```

where:

```
T_stress_p(q) = 0.5 * sum_j ( sigma_1(p,j) + sigma_2(p,j) ) * n_hat(j)
T_correction_p(q) = -penalty * (u1_p(q) - u2_p(q) - f_p(q))
```

This is computed in `ComputeTractionAtQuadPoints()` (`dg_elasticity_ip_combined_integrator.hpp:421-`

```
// Stress average: {sigma} = 0.5*(sigma_0 + sigma_1)
for (int p = 0; p < dim_; p++) {
   real_t T_p = 0.0;
   for (int j = 0; j < dim_; j++) {
      real_t eps1 = 0.5*(grad1(p,j) + grad1(j,p));
      real_t eps2 = 0.5*(grad2(p,j) + grad2(j,p));
      real_t sig1 = (p==j ? lam1t*tr1 : 0.0) + 2.0*mu1t*eps1;
      real_t sig2 = (p==j ? lam2t*tr2 : 0.0) + 2.0*mu2t*eps2;
```

```
      T_p += 0.5 * (sig1 + sig2) * n_hat(j);
   }
   traction_q(p * nq + q) = T_p;
}
// Penalty correction: -penalty * (u_jump - slip)
traction_q(c * nq + q) += (-penalty) * (u1q - u2q - f_q);
```

Projection to fault DOFs:

The 3D traction at quad points is projected to 2-component (dip, strike) values at fault DOFs using `ProjectTractionToFaultDOFs()`:

```
trac_dof[comp, k] = sum_q (T(q) . tangent_comp(q)) * basis_func[k,q] * |n(q)| * w_q
```

where `tangent_comp` is the dip or strike tangent vector, and `|n(q)|` is the face-area Jacobian at each quad point.

6.4 Normal Traction (sigma_n Feedback)

For the elastic normal stress feedback (v51/v55), the normal component of traction is also extracted:

```
T_n(q) = sum_p T_p(q) * n_hat_p(q)
```

This is stored in `normal_traction_` and passed to `ComputeRHS()` to modify the effective normal stress: `sigma_n_eff = sigma_n + T_n` (matching Tandem's `DieterichRuinaBase.h:87`).

---

7. Rate-and-State Friction and State Evolution

7.1 Regularized Dieterich-Ruina Friction

File: `friction/dieterich_ruina.hpp`

The friction coefficient in psi-space (regularized form):

```
f(V, psi) = a * asinh( (V / (2*V0)) * exp(psi / a) )
```

where psi = f0 + b * ln(V0 * theta / Dc) is the logarithmic state variable.

This is evaluated in `FrictionCoefficientPsi()` (`dieterich_ruina.hpp:~316`).

7.2 Stress Balance: Solving for V

The friction law defines the stress balance:

```
|tau| = sigma_n * f(|V|, psi) + eta * |V|
```

Given tau (from domain solve + pre-stress) and psi (from state), we solve for |V| using Brent's method (root bracketing):

```
F(V) = sigma_n * a * asinh(V/(2V0) * exp(psi/a)) + eta*V - |tau| = 0
```

This is monotonically increasing in V, guaranteeing a unique root.

Scalar solver: `SolveSlipRatePsi()` (`dieterich_ruina.hpp:~350`) uses Brent's `zeroIn` (ported from Tandem).

Vector solver: `SolveSlipRateVectorPsi()` (`dieterich_ruina.hpp:413-431`): 1. Compute `|tau| = sqrt(tau_dip^2 + tau_strike^2)` 2. Call scalar solver to get `|V|` 3. Decompose: `V = -(|V| / |tau|) * tau` (anti-parallel, matching Tandem)

```
void SolveSlipRateVectorPsi(const real_t tau_vec[2], real_t psi,
                            real_t sigma_n, real_t eta, real_t a,
                            real_t V_vec[2]) const
{
   real_t tau_abs = sqrt(tau_vec[0]*tau_vec[0] + tau_vec[1]*tau_vec[1]);
   real_t V_abs = SolveSlipRatePsi(tau_abs, psi, sigma_n, eta, a);
   V_vec[0] = -(V_abs / tau_abs) * tau_vec[0];
   V_vec[1] = -(V_abs / tau_abs) * tau_vec[1];
}
```

7.3 State Evolution

File: `friction/state_evolution.hpp`

The aging law in psi-space:

```
dpsi/dt = (b * V0 / Dc) * exp(-psi / b) - (b / Dc) * |V|
```

This is called from `RateStateFaultOperator::ComputeRHS()` at `rate_state_fault.hpp:463`:

```
rate(i * StatePerNode + PsiIndex) = evolution_->Rate(V_abs, psi, Dc);
```

7.4 The Full ComputeRHS Loop

`RateStateFaultOperator<MeshType, 2>::ComputeRHS()` (`rate_state_fault.hpp:363-469`) is the core coupling point. For each fault DOF i:

```
// Total stress = pre-stress + elastic traction
real_t tau_vec[2] = {tau_pre_(2*i) + traction(2*i),
                     tau_pre_(2*i+1) + traction(2*i+1)};

// Effective normal stress (with elastic feedback)
real_t sigma_n_eff = sigma_n_bp5_ + (*normal_traction)(i);

// Solve friction for slip rate vector
real_t V_vec[2];
dr_friction_->SolveSlipRateVectorPsi(tau_vec, psi, sigma_n_eff, eta, a, V_vec);
real_t V_abs = sqrt(V_vec[0]*V_vec[0] + V_vec[1]*V_vec[1]);

// Set rates: d(slip)/dt = V, d(psi)/dt = aging law
```

```
rate(i * StatePerNode + 0) = V_vec[0];       // dip slip rate
rate(i * StatePerNode + 1) = V_vec[1];       // strike slip rate
rate(i * StatePerNode + PsiIndex) = evolution_->Rate(V_abs, psi, Dc);
```

7.5 Pre-stress and Initial Conditions

Pre-stress tau_pre is computed per-DOF in `FaultGeometry::ComputeBP5Params()` using `BP5Params::tau0_vec()` (`bp5_params.hpp:333-381`):

```
tau0 = sigma_n * a * asinh(V_init/(2V0) * exp(psi_ss/a)) + eta * V_init
```

where `psi_ss = f0 + b * ln(V0 / Vp)` is the steady-state psi at plate rate. In the nucleation zone, an extra `delta_tau = delta_tau_factor * eta * V_nuc` is added (SCEC Eq. 23).

Initialization follows Tandem's two-phase approach (`seas_operator.hpp:184-241`): 1. `PreInit`: set slip=0, psi=placeholder (steady-state at V_init) 2. Solve K u = b with zero slip to get initial elastic traction 3. `Init`: compute psi from stress equilibrium using `InitialStatePsi()` 4. Verify: re-solve and check that stress equilibrium error $<$ 1e-6

7.6 State Layout

The ODE state vector has 3 components per fault DOF (SlipComponents=2):

```
state = [s_dip_0, s_strike_0, psi_0, s_dip_1, s_strike_1, psi_1, ...]
         |<---- DOF 0 ---->|           |<---- DOF 1 ---->|
```

Total state size = 3 * num_fault_dofs. The ODE integrates all components simultaneously: d(state)/dt = [V_dip, V_strike, dpsi/dt].

---

8. Time Integration

8.1 Dormand-Prince RK45

File: `solver/time_stepper.hpp`

The code uses a Dormand-Prince 5(4) embedded Runge-Kutta method with 7 stages and FSAL (First Same As Last). The 5th-order solution advances the state; the 4th-order embedded solution provides a local truncation error estimate.

Step structure (`time_stepper.hpp:227-600`):

```
Stage 0: k[0] = f(t, state)                    (reused from previous step via FSAL)
Stage 1: k[1] = f(t + c2*dt, state + dt*a21*k[0])
Stage 2: k[2] = f(t + c3*dt, state + dt*(a31*k[0] + a32*k[1]))
...
Stage 5: k[5] = f(t + c6*dt, state + dt*(a61*k[0] + ... + a65*k[4]))
Stage 6: k[6] = f(t + dt, y_new)               (FSAL: becomes k[0] next step)
```

```
y_new = state + dt * (b1*k[0] + b3*k[2] + b4*k[3] + b5*k[4] + b6*k[5])
error = dt * (e1*k[0] + e3*k[2] + e4*k[3] + e5*k[4] + e6*k[5] + e7*k[6])
```

Each `f(...)` evaluation calls `SEASQuasiDynamicOperator::Mult()`, which triggers a full domain solve.

8.2 Adaptive Step Size Control

The error norm uses L-infinity (matching Tandem/PETSc `wnormtype infinity`):

```
err_i = |error_i| / (atol + rtol * |state_i|)
err_norm = max_i(err_i)    (global across MPI ranks)
```

Accept/reject decision: - If `err_norm <= 1.0`: accept step, advance state and time - If `err_norm > 1.0`: reject step, shrink dt

Step size update:

```
factor = safety * err_norm^(-1/5)    (5th order method)
factor = clamp(factor, shrink_min, growth_max)
dt_new = dt * factor
```

After rejection, an extra `reject_safety` factor (0.5) is applied.

Default tolerances (matching Tandem): - `atol = 1e-7` - `rtol = 1e-50` (effectively pure absolute tolerance) - `dt_min = 1e-6 s`, `dt_max = 0.5 year`

8.3 V Guard

A stage-level velocity guard (`time_stepper.hpp:253-269`) rejects steps if any intermediate RK stage produces slip rates exceeding `V_guard_factor * V_max_stage0`. This catches CFL-violating steps before they produce NaN.

---

9. Parallelism

9.1 Domain Decomposition

The mesh is distributed via MFEM's `ParMesh`. Each rank owns a subset of elements. The DG FE space (`ParFiniteElementSpace`) manages DOF numbering.

9.2 Shared Fault Faces

Fault faces at MPI partition boundaries appear as "shared faces" rather than interior faces. The code tracks them separately:

- `fault_interior_faces_`: fault faces where both elements are local
- `fault_shared_faces_`: fault faces at partition boundaries

Shared faces are handled by separate assembly loops that use `GetSharedFaceTransformations()` and face-neighbor element data.

9.3 Stiffness Assembly in Parallel

`ParBilinearForm::Assemble()` handles shared face assembly automatically. `ParBilinearForm::ParallelAssemble()` creates a `HypreParMatrix`.

The slip RHS for shared faces is assembled in `AssembleSlipContributionIPShared()` (`elasticity_operator.hpp:1544-1640`): only the local element's contribution is assembled (element 2 is a ghost from the neighbor rank).

9.4 Key MPI Communication Points

Operation	Where	MPI call
Fault face detection (global tag check)	`BuildFaultTaggedFaces()`	`MPI_Allreduce(MAX)`
Stiffness assembly (shared faces)	`ParBilinearForm::Assemble`	Face neighbor exchange
Linear solve	MUMPS/CG+AMG	Internal MPI in Hypre/MUMPS
Error norm for time stepping	`DormandPrinceRK45::GlobalMax()`	`GlobalMax()` via `MPIContext`
Max slip rate	`GetGlobalMaxSlipRate()`	`GlobalMax()` via `MPIContext`
Probe output ownership	`ParallelBP5BenchmarkOutput`	`MPI_Allreduce(MIN)`

---

10. I/O and Benchmark Output

10.1 SCEC fltst Output

The primary output follows the SCEC SEAS benchmark format: one ASCII file per probe station, with columns for time, slip, slip rate, shear stress, and state variable.

Output architecture (`io/bp5_parallel_output.hpp`): - `ParallelBP5BenchmarkOutput` uses distributed ownership: each rank locates stations on its local fault DOFs, and MPI_Allreduce(MIN) assigns ownership to the rank with minimum geometric distance. - `Probe2DInterpolator` interpolates from face-averaged values or multi-DOF face basis functions to station coordinates. - `ProbeOutput` (`io/probe_output.hpp`) handles individual file writing.

Standard BP5 stations: 10 on-fault stations at specific (x2, x3) coordinates along the fault surface (specified by the SCEC benchmark).

10.2 Checkpoint/Restart

`io/checkpoint.hpp` provides binary checkpoint of the state vector and time, enabling restart of long earthquake cycle simulations.

10.3 ParaView/VTK Output

Optional diagnostic VTK output of displacement, boundary attributes, and fault parameter distributions (a, tau_pre, V_init).

---

11. Key Data Structures

Class	File	Role	Key Members
`BP5Params`	`config/bp5_params.hpp`	All physical parameters + spatial functions	`mu()`, `lambda()`, `eta()`, `a_of_x2_x3()`, `tau0_vec()`, `V_init_vec()`
`ElasticityDomainOperator<MeshType, ElementType>`	`operator/elasticity_operator.hpp`	DG elasticity: K assembly, solve, traction	`mesh_`, `cached_a_`, `solver_`, `fault_interior_faces_`, `fault_basis_`
`RateStateFaultOperator<MeshType, 2>`	`fault/rate_state_fault_ode.hpp`	Fault ODE: traction -> slip rate	`slip_rate_`, `tau_pre_`, `Dc_values_`, `V_init_values_`
`SEASQuasiDynamicSolver<MeshType, DomainOp, FaultOp>`	`solver/sea_solver.hpp`	Explicit ODE coupling	`domain_`, `fault_`, `u_gf_`, `traction_`, `slip_`
`FaultGeometry<MeshType>`	`fault/fault_geometry.hpp`	Fault DOF management + spatial params	`depths_`, `a_values_`, `eta_values_`, `coords_x2_`, `coords_x3_`
`FaultBasis`	`fault/fault_basis.hpp`	Per-face coordinate frames	`basis_[]` with `{normal, tangent1, tangent2, sign_flipped, qp_data}`
`DieterichRuinaFriction`	`friction/dieterich_ruina.hpp`	Friction law + slip rate solver	`SolveSlipRatePsi()`, `SolveSlipRateVectorPsi()`, `InitialStatePsi()`
`AgingLaw` / `AgingLawPsi`	`friction/state_evolution.hpp`	State evolution ODE	`Rate(V, psi, Dc)`
`DormandPrinceRK45`	`solver/time_stepping.hpp`	Adaptive time integrator	`k_[7]`, `err_`, `atol_`, `rtol_`

Class	File	Role	Key Members
`DGElasticityIPOperator`	`integrator/dg_elasticity_ip_operator.hpp`	IP-DG interior face assembly + traction recovery	`AssembleFaceMatrix()`, `AssembleSlipFaceRHS()`, `ComputeTractionAtQuadPoints()`
`DGElasticityBR2Operator`	`integrator/dg_elasticity_br2_operator.hpp`	BR2-DG interior face assembly	`AssembleFaceMatrix()`, `TestNormal()`
`FaceQuadrature`	`fault/face_quadrature.hpp`	Precomputed functions for multi-DOF fault	`InterpolateToQuadPoints()`, `BasisAtQuadPoints()`
`MPIContext`	`common/mpi_context.hpp`	MPI wrappers	`GlobalMax()`, `GlobalSumInt()`, `Barrier()`

---

12. How to Add a New Feature

Adding a New Friction Law

1. Create `friction/my_friction.hpp` inheriting from `FrictionLaw`
2. Implement `FrictionCoefficient()`, `SolveSlipRate()`, and psi-space variants
3. In the BP5 driver, replace `DieterichRuinaFriction` with your class
4. The rest of the pipeline (domain solve, traction recovery) is unchanged

Adding a New Output Quantity

1. Decide where the quantity lives: fault DOFs (`rate_state_fault.hpp`) or domain DOFs (`elasticity_operator.hpp`)
2. Add a getter method to the appropriate operator class
3. In `bp5_parallel_output.hpp`, add a column to the probe output format
4. In the time loop, extract and write the quantity

Adding a New Benchmark Problem

1. Create `config/my_params.hpp` following `bp5_params.hpp` pattern
2. Create a driver in `tests/verification/` following `bp5_verification_full.cpp`
3. Choose the domain operator (`ElasticityDomainOperator` for 3D, `AntiplaneDomainOperator` for 2D antiplane)
4. Set `SlipComponents` template parameter (1 for 2D, 2 for 3D)
5. Wire up the parameter functions (a(x), L(x), tau_pre, etc.) in `FaultGeometry`

---

13. Known Limitations and TODOs

- ☐ The stiffness matrix K is assembled once and cached. This assumes the mesh and material properties do not change. For problems with evolving geometry or nonlinear materials, K would need reassembly.
- ☐ The friction solver uses Brent's method (guaranteed convergence but not optimal speed). Newton-Raphson with Brent fallback could be faster.
- ☐ The BR2 method path uses face-averaged (single-DOF) fault discretization while the IP path supports multi-DOF per face. BR2 multi-DOF is not implemented.
- ☐ Off-fault station output (displacement at interior points away from the fault) is partially implemented but not fully tested in parallel.
- ☐ No dynamic (fully inertial) mode – only quasi-dynamic with radiation damping.
- ☐ The psi-space state evolution (AgingLawPsi) is used for BP5 but the transformation is handled inside the RateStateFaultOperator rather than being a separate evolution law class.

End of guide. Regenerate with: `Use the chunhui-codebase-guide subagent on /Users/chunhuizhao/projects/seas-mfem/miniapps/seas/`